Linux System Administration (E1ITA307)

Linux Infrastructure Deployment Project Report

On

LINUX SYSTEM HEALTH AND PREDICTIVE MONITORING

Submitted by

B.Ritvik-2411021060705

Sahana A Y-2411021060687

T.Jagan Mohan-2411021060957

K.Varun Reddy-2411021060996

In partial fulfillment of the award of the degree of

BACHELOR OF TECHNOLOGY

In

Information Technology

Under the Supervision of

Prof. Kusuma J

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY

BENGALURU

October 2026



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the project work entitled "Linux System Health and Predictive Monitoring", submitted by B.Ritvik-2411021060705, Sahana A Y-2411021060687, T.Jagan Mohan-2411021060957, K.Varun Reddy-2411021060996 in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering of Alliance University, is a bonafide work during the academic year 2026-2027.

_____	_____	_____
Course Handling Faculty	Course Coordinator	Professor and HoD

Date _____

Place _____

Table of Contents

1. Introduction 4
1.1 Aim and 1.2 Background 4
1.3 Problem Statement 4
1.4 Objectives 5
2. Methodology 6
2.1 Tools and Technologies Used 6
2.2 Hardware Requirements and 2.3 Other Resources 7
2.4 Approach and Development Process 7
3. Linux System Administration Design and Implementation 8
3.1 System Architecture 8
3.2 Linux Environment Setup 9
3.3 User and Group Management 10
3.4 File and Directory Management 11
3.5 Package and Process Management 13
3.6 Storage and File System Configuration 14
3.7 Backup and Automation 17
3.8 Security and Access Control 18
4. Results and Testing 19
4.1 Implementation Results 19
4.2 Testing and Validation 21
5. Conclusion and Future Scope 23
References 24
Appendices 25
Appendix A Commands and Reproduction 25
Appendix B Collector Source Excerpt 26
Appendix C Forecasting Source Excerpt 27
Appendix D Alert Source Excerpt 28
Appendix E Dashboard and CSV Source Excerpt 29

1 Introduction

1.1 Aim

The aim of this mini project is to implement Linux System Health and Predictive Monitoring using Bash and standard Linux commands. It collects resource readings, saves a timestamped history, checks usage limits and estimates short-term trends. A local browser dashboard presents the results for system administration and classroom demonstration.

1.2 Background

Linux exposes system information through interfaces such as /proc/stat, /proc/meminfo and /proc/net/dev. Reading these counters at regular intervals makes it possible to calculate utilization and traffic rates. Filesystem usage is available through the df command. [1]

This implementation combines a Bash sampling loop, awk calculations, CSV records and a generated HTML dashboard. Ordinary least-squares regression estimates where recent usage is heading. The estimate can support capacity investigation when the recent workload trend continues.

1.3 Problem Statement

Manual checks require repeated commands and do not automatically preserve observations. High usage may be noticed only after responsiveness changes. A small Linux-command project is needed to collect readings, display history and identify present or possible future threshold crossings.

1.4 Objectives

Collect CPU, memory, root filesystem and network readings using Linux interfaces.

Save timestamped CPU, memory and disk observations in a CSV file.

Display current values and recent history in a local browser dashboard.

Estimate CPU, memory and disk usage 60 seconds ahead using awk.

Show current and qualified predicted warnings and append them to a text log.

Verify collection, calculations, output files and controlled alert behavior.

1.5 Scope of the Implementation

The monitor observes one Linux environment at a time. Bash and standard utilities perform collection and calculations; HTML, CSS and SVG provide the frontend. The dashboard is an automatically regenerated local file. No database service, application server or network port is required.

1.6 Execution Evidence

The report evidence comes from an actual run on 25 October 2026 in a controlled Ubuntu Linux container. It contains 20 readings at a four-second interval. These are separate from the student WSL installation. Command images reproduce captured text, and the browser image displays the HTML produced from those readings.

2 Methodology

2.1 Tools and Technologies Used

The monitoring program runs with Bash and common Linux utilities. It does not require an additional programming runtime or an external database. The frontend contains HTML and CSS with SVG charts; no JavaScript library or internet connection is needed during normal use.

Tool or component	Purpose
Linux and WSL	Linux resource interfaces and classroom execution
Bash	Sampling loop, arguments, signals and file handling
awk	Percentages, rates, regression and HTML rendering
df and /proc files	Disk usage and kernel resource counters
date, sleep and tail	Timestamps, sampling delay and recent history
printf, mktemp and mv	CSV records and complete dashboard replacement
HTML CSS and SVG	Browser layout, automatic refresh and charts

2.1.1 Output Evidence Method

Command-output figures are rendered transcripts of executed commands. Source figures show excerpts of the files used in the run. A browser preview displays the actual generated dashboard. Known input series used to check prediction arithmetic are identified as test fixtures and are not presented as measured resource usage.

2.2 Hardware Requirements

A laptop or desktop with a working Linux environment is sufficient for the project. The classroom installation uses WSL on Windows. The monitor requires readable /proc interfaces, Bash, common utilities and a writable output directory. A browser opens the generated HTML file.

The verification environment exposes container-visible CPU and memory counters. These can describe resources shared with the host rather than a separately limited container. On WSL, the readings describe the Linux environment and should not be interpreted as a complete view of all Windows applications.

2.3 Other Tools and Resources

The project contains linux_health.sh, dashboard.awk, dashboard.template.html and README.txt. A normal run creates health-data/metrics.csv and health-data/dashboard.html. The alerts.log file is created when the first warning occurs. Windows File Explorer can open the project directory from WSL using explorer.exe . [5]

2.4 Approach and Development Process

First, inspect the Linux counters and define the sampling and alert rules. Next, calculate readings and save them as CSV. Maintain the latest 20 observations for regression and render a complete dashboard. Finally, verify the live run, controlled warnings, backup, mathematical fixtures and browser layout.

3 Linux System Administration Design and Implementation

3.1 System Architecture

The system operates locally. Bash reads CPU and network counters before and after a sampling delay. The awk command converts their differences into usage and traffic rates, reads memory information and evaluates resource limits. The df command supplies root filesystem usage.

Each observation is appended to metrics.csv. A temporary history file retains the latest 20 readings from the current run. The dashboard renderer calculates trends, builds SVG chart lines and substitutes the measured values into the HTML template. The browser reloads the generated file every five seconds.

Component	Input	Output
Collector	Linux counters and df	Percentages and network rates
CSV storage	Timestamped observation	Append-only metrics.csv
Trend calculation	Latest 20 readings	Estimate slope and R-squared
Alert checks	Current values and forecasts	Warnings and alerts.log
Dashboard renderer	History and HTML template	Generated dashboard.html

The default sampling interval is five seconds. The captured verification used four seconds and stopped after 20 samples. Complete HTML is written to a temporary file before mv replaces the dashboard, so the browser does not normally read a partially written page.

3.2 Linux Environment Setup

3.2.1 Linux Distribution and Version

The commands below identify the verification environment as Ubuntu 24.04.3 LTS with Linux kernel 6.18.55 and Bash 5.2.21. The root account was supplied by the container environment. It is not the identity of the student Windows or WSL account.

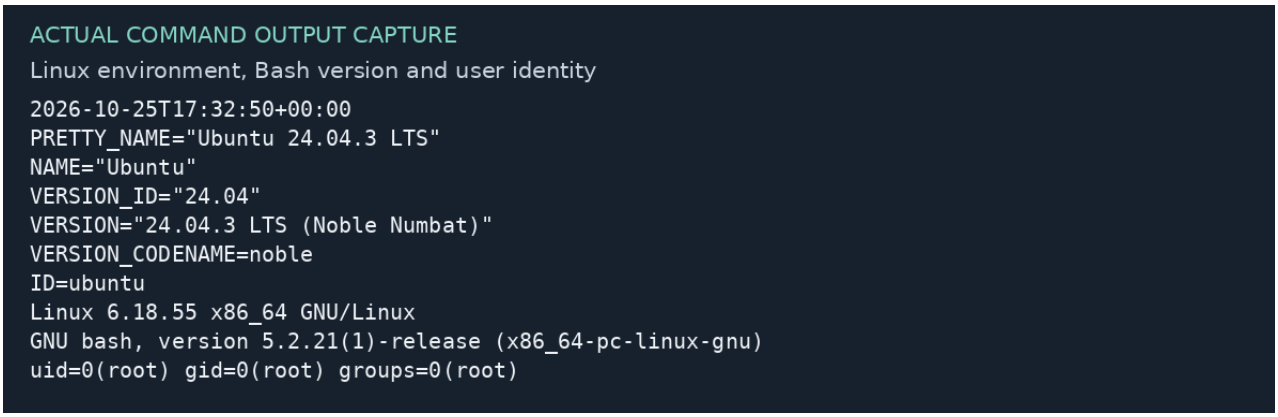


Figure 3.1 Actual Linux environment and Bash version output

3.2.2 Installation and Launch

Extract linux-health-dashboard.tar.gz into the Linux home directory and enter the extracted project folder. Start the monitor with bash linux_health.sh. Open health-data/dashboard.html after the first reading. The runtime dependencies were already available in the captured environment; no installation was performed.

3.3 User and Group Management

3.3.1 User Accounts and Ownership

The project does not create accounts or groups. The captured container supplied the root account shown below. On the student WSL installation, use the existing ordinary Linux account and keep the project in a writable folder under its home directory.

```
ACTUAL COMMAND OUTPUT CAPTURE
Actual account and source file permissions
uid=0(root) gid=0(root) groups=0(root)
-rw-r--r-- root:root linux-command-report/source/linux-health-dashboard/linux_health.sh
-rw-r--r-- root:root linux-command-report/source/linux-health-dashboard/dashboard.awk
```

Figure 3.2 Actual user identity and Bash project file permissions

3.3.2 File Permissions and Access Control

Bash reads linux_health.sh when it is launched with bash linux_health.sh, so the file does not need an executable permission bit for that command. The renderer and template must also be readable. The selected data directory must permit creating and replacing files.

The application does not contain a requirement to run sudo or change ownership. The available verification was performed as the environment root account, so operation under a separate least-privilege service account was not established by this run.

3.4 File and Directory Management

3.4.1 Project Directory Structure

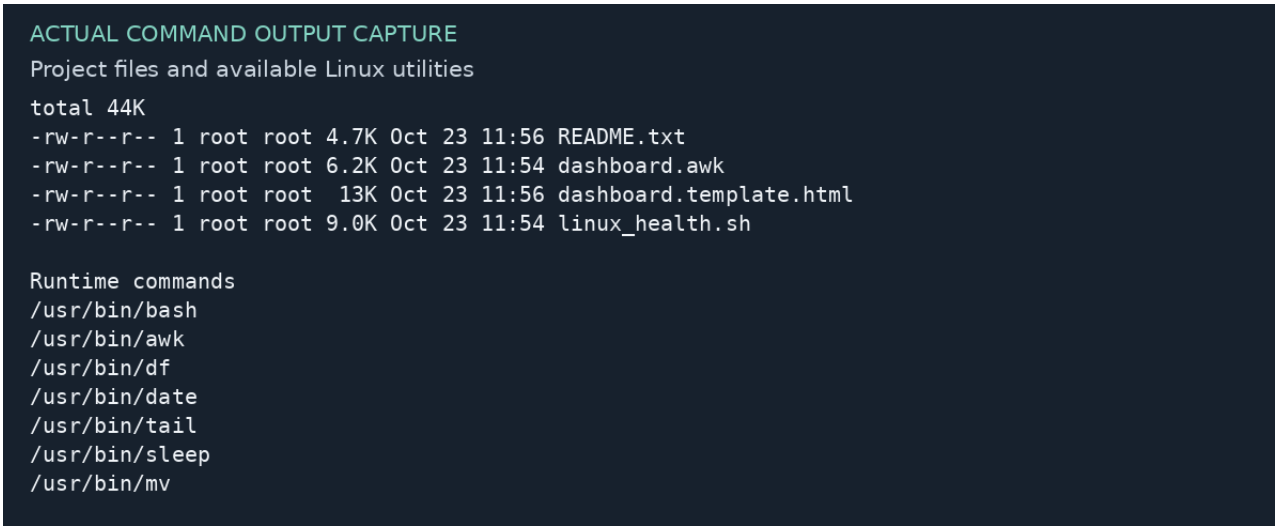


Figure 3.3 Actual project files and available Linux utility paths

linux_health.sh contains the collector, command-line options, terminal output and warning logging. dashboard.awk builds metric cards, charts, forecasts and recent-reading rows. dashboard.template.html defines the frontend layout and refresh instruction. README.txt explains setup and presentation commands.

3.4.2 Runtime Data Locations

The normal output directory is health-data. The --data-dir option selects a different location. This report uses live-data for measured history and demo-data for a deliberately lowered threshold. Separate directories keep demonstration warnings apart from the normal run.

Only one collector should write to a particular output directory at a time. A new run appends to the existing CSV, but its prediction window begins with new readings.

3.4.3 Resource Collection and Inspection

CPU usage uses changes in total and idle counters. Memory usage uses available memory relative to total memory. The df command reports root filesystem percentage at whole-percent resolution. Network rates use byte differences divided by elapsed epoch seconds, excluding the loopback interface.

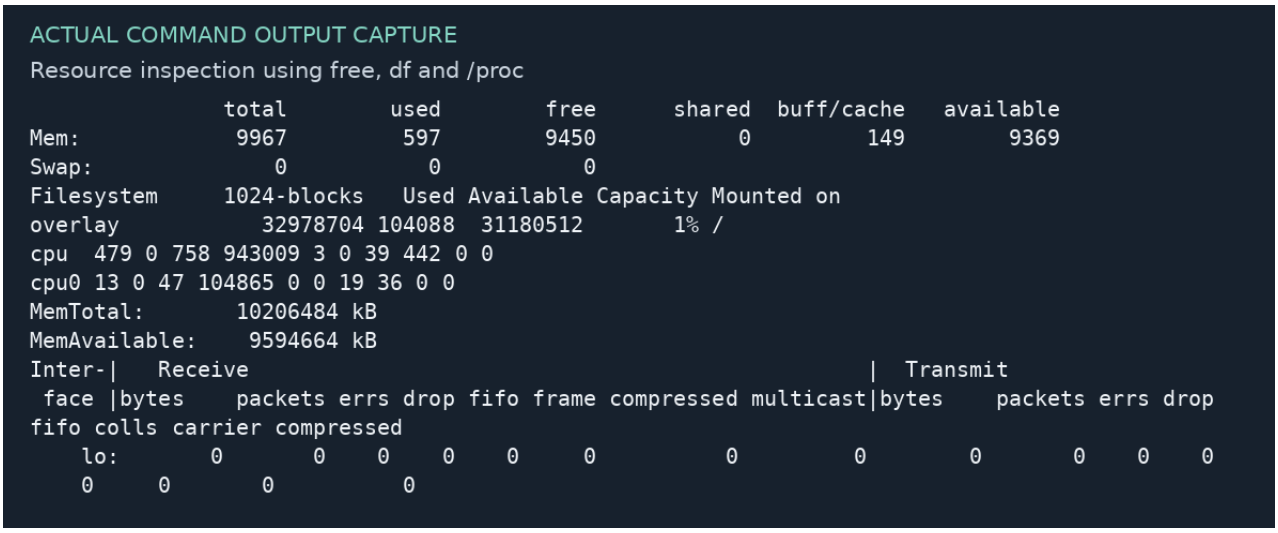


Figure 3.4 Actual Linux resource counters and filesystem command output

The independent free and df commands provide context rather than an exact synchronized comparison. Readings can change between commands. Network rates reset their display when a counter decreases or elapsed time does not advance. Long suspensions and backward clock changes restart the trend history.

3.5 Package and Process Management

3.5.1 Runtime Dependencies

The project uses Bash, awk, df, date, tail, sleep, mkdir, mktemp and mv. These utilities were available in the verification environment. The Bash command orchestrates the project while awk performs decimal arithmetic and text processing. [2]

3.5.2 Application Launch and Resource Logs

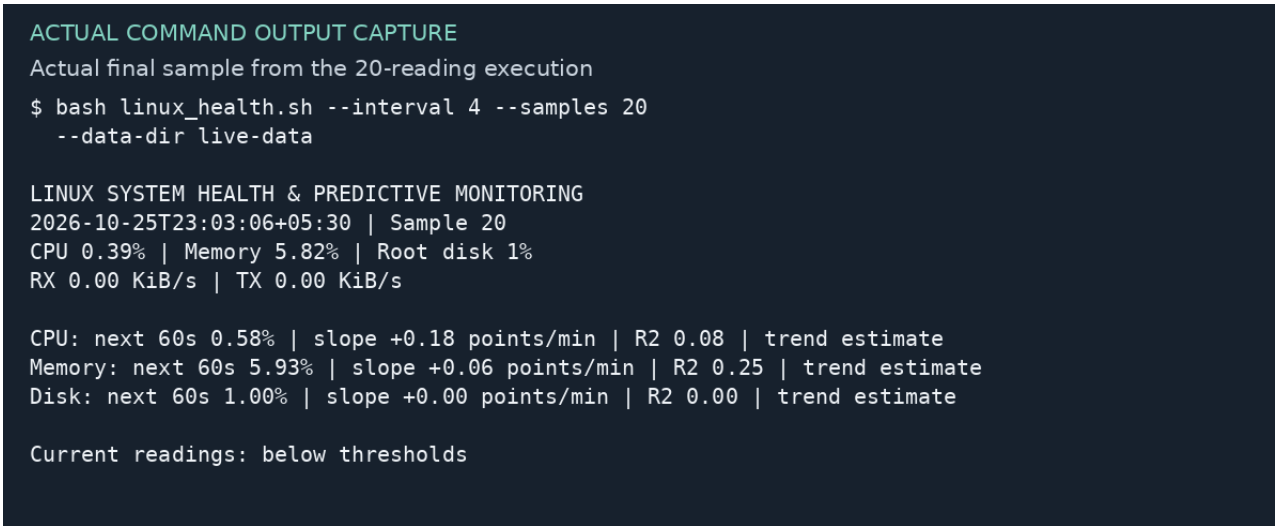


Figure 3.5 Actual Bash launch configuration and final monitoring sample

The foreground loop continues until stopped or until the requested --samples count is reached. The captured run used --interval 4 --samples 20. The normal classroom command omits these options and continues with a five-second delay between readings.

Ctrl+C stops the foreground monitor. The script defines signal and exit traps to remove temporary files. A separate finite-run completion check verified exit status zero and the continued availability of the generated HTML and CSV files.

3.6 Storage and File System Configuration

3.6.1 Existing Filesystem and CSV Storage

The monitor uses the existing filesystem without creating partitions, volumes or mount points. Each CSV row contains epoch, cpu_percent, memory_percent and disk_percent. Network rates appear in the terminal and dashboard but are not stored as columns in this CSV.

```
ACTUAL COMMAND OUTPUT CAPTURE
CSV records and generated dashboard files
$ head -n 4 live-data/metrics.csv
epoch,cpu_percent,memory_percent,disk_percent
1792949509,0.06,5.75,1
1792949513,0.25,5.77,1
1792949517,0.17,5.79,1

$ awk END live-data/metrics.csv (row count)
20 data rows

$ ls -lh live-data
total 24K
-rw-r--r-- 1 root root 17K Oct 25 23:03 dashboard.html
-rw-r--r-- 1 root root 506 Oct 25 23:03 metrics.csv
```

Figure 3.6 Actual CSV record count and generated output files

3.6.2 Retention and File Integrity

The captured CSV contains 20 data records plus one header row. It can be inspected with head, tail and awk. CSV storage avoids a database service and makes the observations easy to open in a spreadsheet.

The implementation has no automatic retention or log rotation. metrics.csv and alerts.log continue growing. For prolonged operation, an administrator should archive or rotate them. A separate stopped-file backup comparison is documented in Section 3.7.

3.6.3 Predictive Monitoring Method

For each resource, awk fits usage = intercept + slope x elapsed time by ordinary least squares. It evaluates the line 60 seconds beyond the latest reading and limits the result to 0 to 100 percent. Prediction requires 20 readings spanning at least 60 seconds. [3]

```
ACTUAL COMMAND OUTPUT CAPTURE
Actual predictions calculated from the measured history
CPU: next 60s 0.58% | slope +0.18 points/min | R2 0.08 | trend estimate
Memory: next 60s 5.93% | slope +0.06 points/min | R2 0.25 | trend estimate
Disk: next 60s 1.00% | slope +0.00 points/min | R2 0.00 | trend estimate
```

Figure 3.7 Actual awk trend estimates from the captured Linux readings

The live history contains 20 samples spanning 77 seconds. The computed 60-second estimates are CPU 0.58 percent, memory 5.93 percent and disk 1.00 percent. These are calculated future estimates, not measurements taken 60 seconds later.

R-squared describes how well the line fits the observed history. CPU and memory fit values in this run were 0.08 and 0.25, indicating weak linear fits. Constant disk usage is represented with R-squared zero. An available forecast does not prove useful future accuracy.

The window contains only the latest 20 observations from the current run. A non-advancing timestamp or a delay greater than the configured interval plus 30 seconds restarts this history. The five-second default interval provides sufficient history after approximately 100 seconds.

3.6.4 Alert Rules and Controlled Test

Default limits are CPU 80 percent, memory 85 percent and disk 90 percent. A current value at or above its limit produces a current warning. A predicted warning requires current usage below its limit, a positive slope, R-squared at least 0.5 and a forecast reaching the limit.

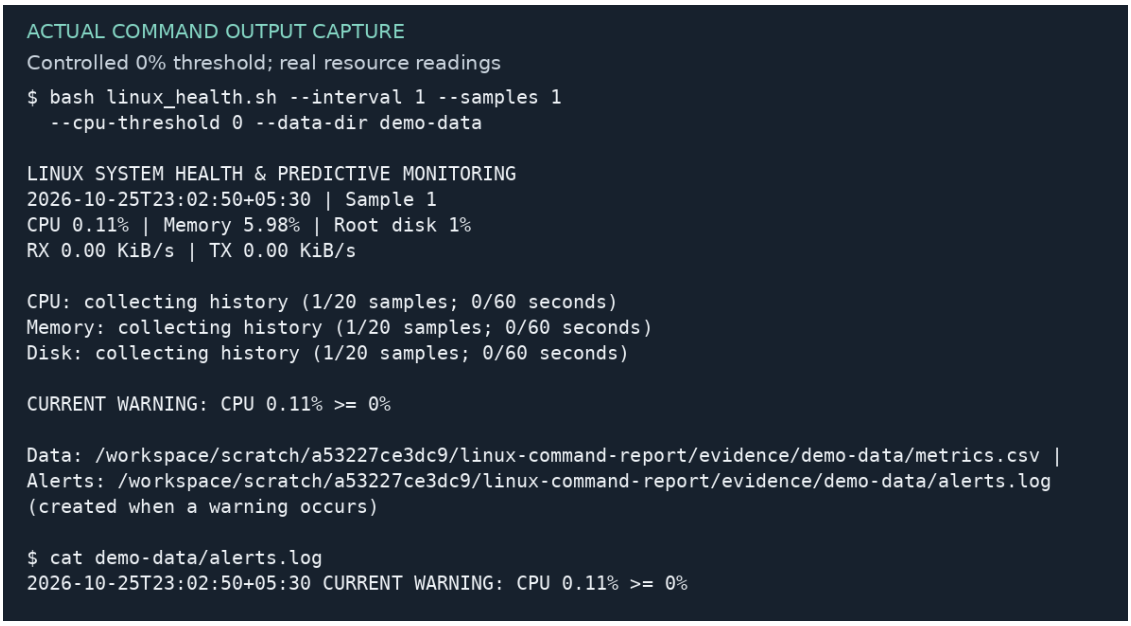


Figure 3.8 Actual controlled CPU warning with a deliberately lowered limit

The demonstration sets the CPU limit to zero and records the real CPU reading without generating artificial load. The dashboard displays a demonstration banner. This verifies the warning path and must not be described as a real resource overload.

Warnings are appended on each qualifying sampling cycle. This version does not suppress repeated warnings or create separate cleared events. The dashboard shows currently active conditions and updates when later readings fall below their limits.

3.7 Backup and Automation

3.7.1 Verified CSV Backup

After the finite collection run stopped, the cp command copied metrics.csv into a backup folder. The cmp command checked that the files matched byte for byte. SHA256 checksums provide a second comparison of the saved bytes. This is a manual administration procedure, not an automatic monitor feature.

```
ACTUAL COMMAND OUTPUT CAPTURE
Stopped-file backup verified with cmp and SHA256
$ cmp live-data/metrics.csv backup/metrics.csv
cmp exit status: 0

$ sha256sum live-data/metrics.csv backup/metrics.csv
213f1eb49d8d652d29a46073520dd8eec76a5909269a9aaed793f4b45d08b430  live-data/metrics.csv
213f1eb49d8d652d29a46073520dd8eec76a5909269a9aaed793f4b45d08b430  backup/metrics.csv
```

Figure 3.9 Actual stopped-file CSV backup and checksum comparison

The comparison returned exit status zero and matching hashes. Copying after collection stopped avoids describing an actively changing CSV as a consistent point-in-time backup. These checks establish equality of the recorded files, not a database integrity guarantee.

3.7.2 Scheduled Operation

The project automates collection with a Bash while loop and sleep. It does not install a cron job or systemd service. The classroom demonstration uses a foreground WSL terminal. Background startup, reboot persistence and scheduled archival can be added and verified as later administration tasks.

3.8 Security and Access Control

3.8.1 Local Dashboard Files

The dashboard opens as a local HTML file. The project does not listen on a network port or provide HTTP endpoints. Bash writes a complete temporary HTML file and then replaces dashboard.html. The browser refresh instruction reloads that file every five seconds.

ACTUAL COMMAND OUTPUT CAPTURE

Actual local-file rendering and refresh commands

```
$ grep -n dashboard rendering and replacement commands
109:    for command in awk df date tail mkdir mktemp rm sleep mv; do
115:    window=$(mktemp)
116:    html_tmp=$(mktemp "$data_dir/.dashboard.XXXXXX")
117:    trap 'rm -f -- "$window" "$html_tmp"' EXIT
118:    trap 'printf "\nMonitor stopped. Readings saved in %s\n" "$data_dir"; exit 0' INT TERM
160:        mv -f -- "$html_tmp" "$data_dir/dashboard.html"

$ grep page refresh and CSV link
<meta http-equiv="refresh" content="5">
href="metrics.csv"
```

Figure 3.10 Actual temporary file replacement and local refresh source

3.8.2 Data Handling and Safe Operation

The collector reads resource counters and filesystem statistics without killing processes or changing resource limits. Demo readings and warnings use a separate directory. File permissions control access to CSV, logs and generated pages.

3.8.3 Current Security Limits

The local files have no application login or encryption layer. Sharing them exposes usage patterns and timestamps. The root-account verification does not establish least-privilege service deployment. Remote sharing or automated remediation would require a separate design and validation.

4 Results and Testing

4.1 Implementation Results

The live verification completed on 25 October 2026. The graph below uses the recorded Linux CSV observations without synthetic additions. The percentages describe the container-visible resource interfaces during the short run.

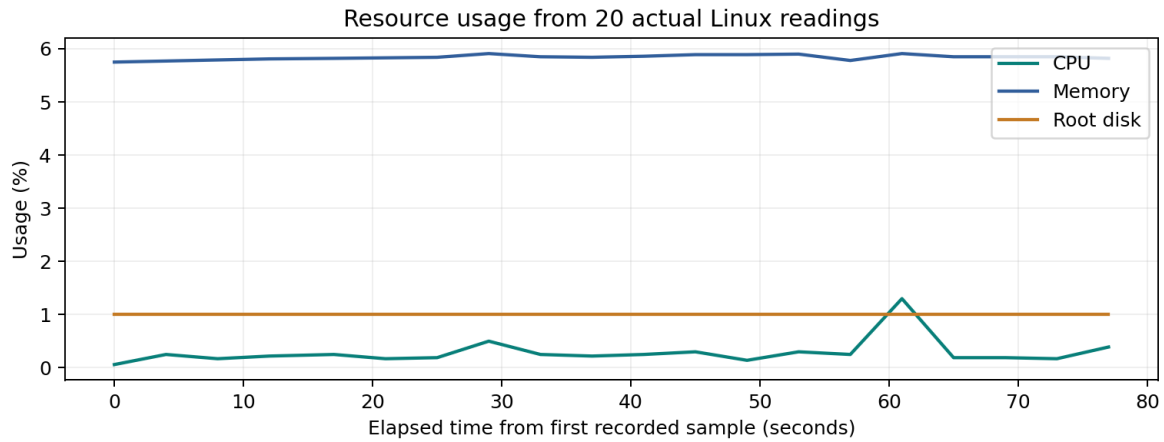


Figure 4.1 Resource usage plotted from 20 actual Linux observations

Result	Observed value
Stored live samples	20
First to last recorded sample	77 seconds
Latest CPU memory and disk	0.39% 5.82% 1.00%
CSV data rows	20
Bash validation checks	10 passed
Dashboard file	Generated and viewed in browser

These results establish collection, persistence and available trend calculations. They do not establish long-term reliability, prediction accuracy or the behavior of the student WSL machine.

4.1.1 Dashboard Output and CSV Results

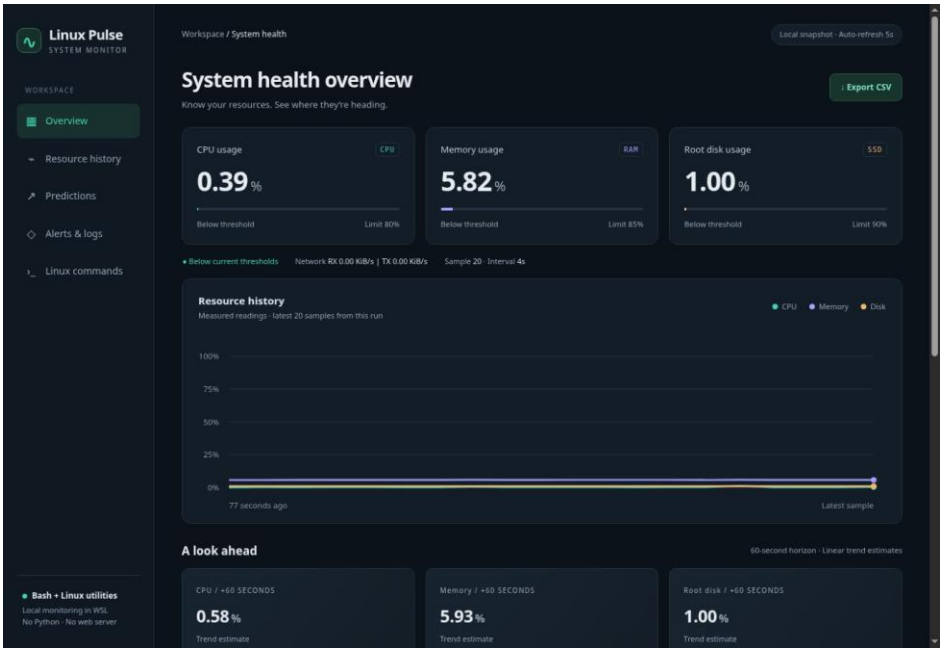


Figure 4.2 Actual browser view of the Bash generated dashboard

```
ACTUAL COMMAND OUTPUT CAPTURE
Actual CSV header and recorded observations
$ head -n 6 live-data/metrics.csv
epoch,cpu_percent,memory_percent,disk_percent
1792949509,0.06,5.75,1
1792949513,0.25,5.77,1
1792949517,0.17,5.79,1
1792949521,0.22,5.81,1
1792949526,0.25,5.82,1

$ tail -n 3 live-data/metrics.csv
1792949578,0.19,5.85,1
1792949582,0.17,5.85,1
1792949586,0.39,5.82,1
```

Figure 4.3 Actual CSV header and saved observations from the Linux run

The browser shows the metric cards, measured resource history and prediction cards generated from the same CSV window. For this report, a local preview rendered the generated HTML for capture; a preview server is not part of the delivered monitor. The Export CSV link targets the saved metrics.csv file.

4.2 Testing and Validation

The following results distinguish live integration checks, controlled threshold conditions and known-input calculation checks. Each passed check establishes its stated behavior in the captured Linux environment.

Test case	Evidence or actual result	Status
Live collection	20 readings spanning 77 seconds	Passed
Forecast warmup	Predictions available on the final live sample	Passed
Generated dashboard	Real HTML viewed in a browser preview	Passed
CSV storage	20 numeric rows and one header	Passed
Controlled CPU warning	Real reading with threshold set to 0%	Passed
Manual CSV backup	cmp status zero and matching SHA256	Passed
Finite completion	Exit status zero and saved output files	Passed
Bash validation	10 checks completed successfully	Passed
Student WSL local-file view	Not directly observed in this verification	Not verified
Reboot or service startup	No scheduled service is implemented	Not tested

Table 1 Testing and validation results

No forecast accuracy percentage or future error score is claimed. Rendering the dashboard in a preview establishes its appearance for the generated HTML; it does not establish the student browser file association or WSL interoperability settings.

4.2.1 Automated Checks

```
ACTUAL COMMAND OUTPUT CAPTURE
Actual Bash validation; known inputs labeled as fixtures
PASS Bash syntax check
PASS One real Linux sample written to CSV
PASS Zero-percent threshold logs a real reading
PASS Dashboard explains the controlled alert demonstration
PASS All dashboard template fields replaced
PASS Insufficient history keeps the prediction unavailable
PASS Known rising test series predicts 41.00 percent and warns
PASS Flat test series remains 40.00 percent without a warning
PASS Prediction is bounded at 100 percent
PASS Dashboard renderer recognizes the known predictive warning

10 checks passed.
Known input series above are test fixtures, not live measurements.
```

Figure 4.4 Actual Bash validation output with test fixtures identified

The Bash verification checks syntax, one real Linux sample, CSV writing, low-threshold logging, the demonstration banner and complete template substitution. Known input series separately check minimum history, a rising predictive warning, flat usage, a bounded forecast and the dashboard warning branch.

The increasing series has a known result of 41.00 percent at the forecast horizon. This number is a deliberate test fixture and is not a measured CPU value from the live run. Synthetic test inputs are kept separate from the evidence CSV and dashboard screenshot.

4.2.2 Testing Summary

All ten checks passed. The live run also established the final prediction output, saved history and browser rendering. Long-term forecast performance, WSL file-opening behavior on the student laptop and automatic startup remain outside the observed results.

5 Conclusion and Future Scope

5.1 Conclusion

The Linux System Health and Predictive Monitoring project combines Bash and standard Linux commands with a local frontend. It calculates CPU, memory, disk and network values, records observations as CSV and generates a dashboard with measured history and configurable warnings.

The captured evidence confirms 20 live observations spanning 77 seconds, available trend estimates, a browser-rendered dashboard and a successful manual CSV backup. A separate zero-limit test verifies actual warning recording. Ten Bash validation checks support specific calculation and output behaviors.

The project demonstrates Linux administration concepts such as reading /proc, using pipes and redirection, managing directories, inspecting permissions, handling signals and backing up files. Its trend estimates support investigation but do not predict hardware failures or establish future accuracy from this short run.

5.2 Future Scope

Future improvements can add log rotation, retained network-rate columns, duplicate-warning suppression and clear events. Scheduled startup and backup could use cron or a verified user service. Forecast evaluation should compare each estimate with a later observation. Additional extensions include per-process metrics, notifications and multiple-machine monitoring.

References

[1] Linux Kernel Documentation. The proc Filesystem. https://www.kernel.org/doc/html/latest/filesystems/proc.html. Accessed 25 October 2026.

[2] GNU Project. Bash Reference Manual. https://www.gnu.org/s/bash/manual/bash.html. Shell scripting reference.

[3] GNU Project. The GNU Awk User Guide. https://www.gnu.org/software/gawk/manual/gawk.html. Text processing and arithmetic reference.

[4] GNU Project. GNU Coreutils Manual. https://www.gnu.org/software/coreutils/manual/. File and system utility reference.

[5] Microsoft Learn. Working across Windows and Linux file systems. https://learn.microsoft.com/en-us/windows/wsl/filesystems. Accessed 25 October 2026.

[6] Project source. linux_health.sh, dashboard.awk, dashboard.template.html and README.txt from linux-health-dashboard.tar.gz.

[7] Execution evidence. Captured command transcripts, metrics.csv, generated dashboard, Bash verification and file backup comparison, 25 October 2026.

Evidence Interpretation

Command-output figures reproduce executed text; source figures show the implementation used for the run. The history plot uses measured CSV values. The dashboard image displays the generated output in a browser preview. Controlled threshold tests and known-input checks are identified separately. The verification environment is a Linux container, not the student WSL machine.

Appendices

Appendix A Commands and Reproduction

Run the commands below in the WSL terminal. Extract the downloaded archive once. Subsequent demonstrations only require entering the folder and restarting the Bash collector. explorer.exe opens Windows File Explorer; select health-data/dashboard.html after the first reading.

```
tar                      -xzf                      /mnt/c/Users/bezaw/Downloads/\
linux-health-dashboard.tar.gz                         -C                          ~
cd                                                       ~/linux-health-dashboard
explorer.exe                                                                     .
bash                                                                linux_health.sh

#           Actual           finite           verification           configuration
bash      linux_health.sh       --interval       4       --samples       20       \
  --data-dir                                                                live-data

#                     View                      saved                      readings
tail                 -n                 5                 health-data/metrics.csv

#                   Safe                    warning                    demonstration
bash           linux_health.sh            --cpu-threshold            0            \
  --data-dir demo-data
```

Keep the terminal open while presenting. Wait about 100 seconds for normal five-second sampling to produce forecasts. Ctrl+C stops collection. For the demo command, open demo-data/dashboard.html and explain the deliberately lowered threshold.



Figure A1 Actual finite Bash run exit status and saved output checks

Appendix B Collector Source Excerpt

This excerpt comes from the executed linux_health.sh. Bash reads successive CPU and network counters. awk performs decimal calculations, and df supplies root filesystem usage. The resulting CSV row contains the epoch timestamp and three percentages.

ACTUAL COMMAND OUTPUT CAPTURE

Actual Bash collector source excerpt

```
        read -r total idle < <(cpu_ticks)
        read -r rx tx < <(network_bytes)
        cpu=$(awk -v dt="$((total-prev_total))" -v di="$((idle-prev_idle))" \
            'BEGIN {v=(dt>0?100*(dt-di)/dt:0); if(v<0)v=0; if(v>100)v=100; printf "%.2f",v}')
        mem=$(awk '/^MemTotal:/ {t=$2} /^MemAvailable:/ {a=$2}
            END {if(t<=0)exit 1; printf "%.2f",100*(t-a)/t}' /proc/meminfo)
        disk=$(df -P / | awk 'NR==2 {gsub(/%/,"",$5); print $5}')
        [[ $disk =~ ^[0-9]+$ ]] || { echo 'Could not read root disk usage.' >&2; exit 1; }
        rates=$(awk -v dr="$((rx-prev_rx))" -v dt="$((tx-prev_tx))" -v sec="$((now-
prev_time))" \
            'BEGIN {if(sec<=0 || dr<0 || dt<0) print "counter reset / clock changed";
             else printf "RX %.2f KiB/s | TX %.2f KiB/s",dr/sec/1024,dt/sec/1024}')
        if ((now<=prev_time || now-prev_time>interval+30)); then
            : > "$window"  # Restart trend history after a clock jump or long suspension.
        fi
        printf '%s,%s,%s,%s\n' "$now" "$cpu" "$mem" "$disk" >> "$csv"
        printf '%s,%s,%s,%s\n' "$now" "$cpu" "$mem" "$disk" >> "$window"
        history=$(tail -n 20 "$window")
        printf '%s\n' "$history" > "$window"
```

Figure B1 Actual Bash source excerpt of resource collection and CSV storage

The CPU helper totals the first eight counters to avoid counting guest time twice. Memory usage is based on MemAvailable. Network rates exclude loopback and are displayed in KiB per second. A temporary history file retains the latest 20 samples for the current trend window.

The timestamp uses wall-clock epoch seconds. Backward time or a long gap resets trend history. This implementation does not use a separate monotonic clock, so substantial forward clock adjustments can affect a rate interval.

Appendix C Forecasting Source Excerpt

The dashboard.awk model function fits usage against elapsed seconds using least squares. It waits for 20 readings over at least 60 seconds, computes slope and intercept, and evaluates the line 60 seconds beyond the latest observation.

ACTUAL COMMAND OUTPUT CAPTURE

Actual awk regression source excerpt

```
function model(c, i,x,y,sx,sy,sxx,sxy,syy,dx,dy,b) {
    if(n<20 || times[n]-times[1]<60) return 0;
    for(i=1;i<=n;i++) {
        x=times[i]-times[1]; y=value[c,i];
        sx+=x; sy+=y; sxx+=x*x; sxy+=x*y; syy+=y*y;
    }
    dx=n*sxx-sx*sx; dy=n*syy-sy*sy;
    if(dx<=0)return 0;
    slope[c]=(n*sxy-sx*sy)/dx;
    b=(sy-slope[c]*sx)/n;
    predicted[c]=b+slope[c]*(times[n]-times[1]+60);
    if(predicted[c]<0)predicted[c]=0; if(predicted[c]>100)predicted[c]=100;
    r2[c]=(dy>0 ? (n*sxy-sx*sy)^2/(dx*dy) : 0);
    if(r2[c]>1)r2[c]=1;
    return 1;
```

Figure C1 Actual awk source excerpt of linear trend forecasting

Predictions are limited to the range 0 to 100 percent. Constant usage yields R-squared zero in this implementation. CPU, memory and disk use the same model, while network traffic is shown as a current rate without a forecast.

The terminal forecast function and dashboard model use the same formula. Known input checks establish selected arithmetic and warning cases. They do not measure prediction accuracy under changing workloads.

Appendix D Alert Source Excerpt

The Bash collector compares each current resource value with its configured limit. The forecast output can also contain a predicted warning. Matching warning lines are timestamped and appended to alerts.log in the selected output directory.

ACTUAL COMMAND OUTPUT CAPTURE

Actual Bash threshold and log source excerpt

```
        current=$(awk -v c="$cpu" -v m="$mem" -v d="$disk" -v cl="$cpu_limit" -v
ml="$mem_limit" -v dl="$disk_limit" \
            'BEGIN {if(c>=cl)printf "CURRENT WARNING: CPU %.2f%% >= %s%%\n",c,cl;
                    if(m>=ml)printf "CURRENT WARNING: Memory %.2f%% >= %s%%\n",m,ml;
                    if(d>=dl)printf "CURRENT WARNING: Disk %.2f%% >= %s%%\n",d,dl}')
        predictions=$(forecast "$window" 2 "$cpu_limit" CPU
                      forecast "$window" 3 "$mem_limit" Memory
                      forecast "$window" 4 "$disk_limit" Disk)
        warnings=$(printf '%s\n%s\n' "$current" "$predictions" | awk '/WARNING/')
        if [[ -n $warnings ]]; then
            while IFS= read -r warning; do printf '%s %s\n' "$(date -Is)" "$warning" >>
"$log"; done <<< "$warnings"
        fi
```

Figure D1 Actual Bash source excerpt of warning checks and alert logging

The predictive branch requires a rising slope, R-squared at least 0.5 and an estimate reaching the threshold while the current value is still below it. A current warning therefore takes priority for that resource.

Every qualifying cycle can append another warning line. Duplicate suppression and explicit cleared events are future improvements. The zero-percent threshold example verifies current-warning logging with a real reading; a known increasing series verifies the predictive branch separately.

Appendix E Dashboard and CSV Source Excerpt

Bash appends each real observation to the CSV file and invokes dashboard.awk with the current limits, interval and history. The renderer substitutes metric values, SVG chart paths, forecasts and warning rows into the HTML template. mv replaces dashboard.html after rendering completes.

ACTUAL COMMAND OUTPUT CAPTURE

Actual CSV append, HTML generation and refresh source

```
        printf '%s,%s,%s,%s\n' "$now" "$cpu" "$mem" "$disk" >> "$csv"
        awk -v cpu_limit="$cpu_limit" -v mem_limit="$mem_limit" -v disk_limit="$disk_limit" \
            -v interval="$interval" -v count="$count" -v generated="$now" \
            -v template="$script_dir/dashboard.template.html" \
            -v rate_text="$rates" -v readable_time="$(date -Is)" \
            -f "$script_dir/dashboard.awk" "$window" > "$html_tmp"
        mv -f -- "$html_tmp" "$data_dir/dashboard.html"

<meta http-equiv="refresh" content="5">
href="metrics.csv"
```

Figure E1 Actual source excerpt of local dashboard generation and CSV export

The refresh meta tag reloads the generated file every five seconds. The Export CSV link points to metrics.csv in the same output directory. There is no local HTTP handler or JSON endpoint in this version. Browser handling of a local download link may open the CSV instead of downloading it.

When the collector stops, the browser retains its last snapshot. The timestamp shows when the readings were generated; page refresh alone does not collect new data. Restart the Bash script to resume sampling.